

10th International Conference of Information and Communication Technology (ICICT-2020)

FPGA autotuning optimization based on structural equation modeling

Yonghao Xu, Ning Xu*

School of Computer Science and Technology, Wuhan University of Technology University, Wuhan Hubei 430070, China

Abstract

DATuner is an autotuning program, but its running time is very long and uneven. In our paper, a novel methodology for autotuning and exploring parameter space is proposed. Combined with Structural Equation Model and statistical techniques, the prior pruning was proposed. Firstly, using benchmarks in DATuner, VTR (Verilog to Routing) was run to generate training data set; Secondly, with training data set as input, modeling software — AMOS was run to construct structural equation model, and then model was validated and evaluated. If the fitness of the model does not reach requirement, then the connections between variables are modified to get final model whose fitness is sufficient. Finally, the parameters' correlation was analyzed which resulted in routing failure. Applying the prior pruning to DATuner, its autotuning time is reduced and running efficiency has been improved. Compared with unmodified DATuner, DATuner with prior pruning decreases by an average of 44% on the autotuning time. The proposed method greatly improves autotuning efficiency. In addition, DATuner's framework was optimized.

© 2021 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 10th International Conference of Information and Communication Technology.

Keywords: Field Programmable Gate Array (FPGA); parameter space; placement; routing; Structural Equation Model (SEM); autotuning;

1. Introduction

In the design and implementation of FPGA(Field Programmable Gate Array), circuits are generally designed and simulated using EDA(Electronics Design Automation) tools. The design flow of FPGA circuit includes synthesis, mapping, placement, routing, simulation, validation and bitstream generation¹. Pack, placement and routing are the three key steps in the design flow, they determine the routability and performance of the final FPGA design. Lots of EDA tools put main optimizations in these three steps. Some commercial EDA tools such as Vivado and Quartus support the complete design flow, but they are not open-source. For promoting the progress of FPGA academic

* Corresponding author. Tel.: 15207177456.

E-mail address: xuyonghao95@qq.com

research, many researchers have developed open-source EDA tools for the convenience of other researchers such as VTR^{1,2} and DATuner. VTR implements synthesis, mapping, placement and routing. DATuner is an open-source FPGA autotuning tool, it integrates VTR.

There are many parameters used as optimization options provided to users in EDA tools. Users can get FPGA design that has good performance and can be routed successfully by setting these parameters reasonably. Every parameter can be discrete, continuous or enumerated value. Although the number of parameters in DATuner is only 25, the parameter combinations in the parameter space are petascale level, it is impossible to traverse such an enormous and complex parameter space³. Based on this problem, researchers proposed autotuning. DATuner is an open-source software that implements autotuning using heuristic algorithm and parameter space partitioning².

However, in the process of running DATuner, it is found that the running time is very long and uneven for various parameter combinations. Sometimes it can converge to an optimal solution in a few minutes; sometimes it can't get a feasible solution even in a few hours. By analyzing a large amount of experimental data, we found that DATuner can't route successfully for every parameter combination for the same FPGA design. In the data set generated by the experiments, a large number of routing failures occurred, accounting for about 72%. Little research has been devoted to routability of the FPGA design. If the parameter combination results in routing failure, then this execution is not necessary. Based on this, we proposed prior pruning based on structural equation model that can reduce running time and increase execution efficiency of DATuner.

2. VTR and OpenTuner in DATuner

DATuner integrates VTR⁴ and OpenTuner⁵. VTR is used to execute FPGA design flow and OpenTuner is an autotuning framework. The following is a brief introduction to VTR and OpenTuner.

2.1. FPGA design flow in VTR

As mentioned above, VTR implements the encapsulation of the FPGA design flow. DATuner calls VTR in the scripts and passes parameters to it. VTR mainly includes the following parts:

- 1) Odin II——front-end synthesis tool, used to parse HDL(Hardware Description Language) file, and converts the HDL file to primary netlist file⁶;
- 2) ABC——logic optimization and mapping tool, perform logic optimization on the netlist file generated in the previous step, and map the logic function to the implementation of LUT(Look-Up Table) and DFF(D Flip-flop)⁷;
- 3) VPR——placement and routing tool, execute pack, placement and routing. The pack process is implemented by the sub-module T-VPACK of the VPR.

The execution results of these three steps will determine whether the FPGA design can be routed successfully. In addition, the latest version of VPR used in DATuner also supports performance evaluations such as timing analysis, area and power estimation^{4,8}. Fig. 1 shows the above flow chart.

2.2. OpenTuner and multi-armed bandit

MAB(Multi-Armed Bandit) model is widely used in statistics and machine learning. MAB originated from a game. The game participants select an arm randomly at each round of the game and obtain corresponding rewards. The reward value of each arm at each round is not fixed and unknown. The goal of the game is to maximize the reward value within a limited number of rounds. As the game progresses, participants can learn the probability distributions and expectations of the reward of each arm through the reward values already obtained in the game. The learned probability distributions and expectations can be used to guide subsequent selections², that is, the higher the reward of an arm, the greater the chance of the arm being selected. At the same time, due to the uncertainty of the rewards, participants should also choose arms with lower reward. This is the 'exploitation-exploration' tradeoff of the MAB model. The research on the MAB problem is all about how to make tradeoff between them.

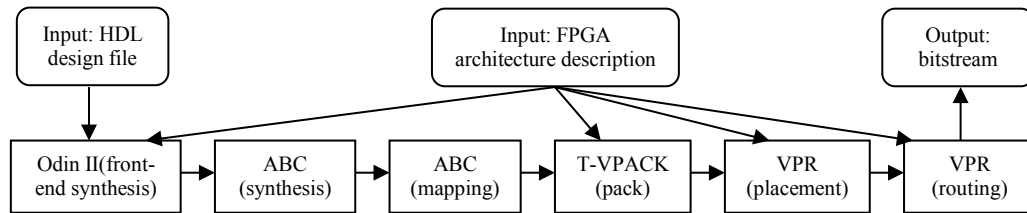


Fig. 1. VTR execution flow

When exploring parameter space, using one particular algorithm is not feasible. So OpenTuner incorporates a set of search techniques, and each search technique is regarded as an arm. DATuner integrates OpenTuner as its core search engine.

2.3. Parameters involved in DATuner

As shown in Fig. 1, at each step, there are a few corresponding parameters which can be used by the designers to control the specific execution of each step. Since the pack, placement and routing are the most important to the routability of the design, the parameter space in this paper is based on these three steps. For the other two steps——synthesis and mapping, it is only needed to specify the HDL design file and FPGA architecture description file to them⁶. The ABC step involves circuit optimization, it contains three parameters: resyn, resyn2, resyn3. These three parameters are three commands that are passed to ABC for simplification, synthesis of the logic circuit, removal of the equivalent node in the circuit, and reconstruction of the circuit structure⁷. Table 1 shows the parameters in DATuner and their steps.

3. Prior pruning for parameter space

Section 2 introduces the execution flow of VTR and OpenTuner in DATuner. Through a large number of experiments, it is found that most of the time for program execution is wasted in the case of routing failure because the program usually does more iterations than routing successfully when routing fails. In the data set generated in the experiments, 72% of the parameter space caused routing failure, and DATuner did not deal with this situation.

Based on the above motivations, in order to optimize the process of autotuning, the current research is divided into two directions: heuristic-based algorithm² and machine learning techniques^{9,10,11}. In these existing researches, it is mainly based on the quantitative analysis of parameter space and execution results, and does not qualitatively analyze this problem from the perspective of routability. Experiments show that most of the DATuner's execution results are routing failure. When routing fails, it is impossible to generate an effective circuit, it is meaningless to perform the synthesis, placement and routing steps for these parameter combinations. If the influence of the parameter combination on the routing result can be evaluated or predicted before executing design flow, the parameter combinations that can't result in routing successfully can be filtered out in advance. Based on this idea, we proposed a prior pruning method for the parameter space using structural equation modeling, and the search efficiency of the parameter space has greatly improved.

4. Modeling based structural equation model

In this section, structural equation modeling is used to model the problem based on the methodology and motivation presented in section 3. Mainly including the following three steps:

- 1) Explore CFA(Confirmatory Factor Analysis) submodel;
- 2) Initialize structural submodel;

3) Evaluate the fitness of the model and make corresponding modifications.

Table 1. Parameter description at each step.

Step	Parameters	Description
synthesis	resyn,resyn2,resyn3	logic-synthesis
pack	alpha_clustering,beta_clustering,cluster_seed_type,allow_unrelated_clustering,connection_driven_clustering	For controlling the pack process of the circuit.
placement	alpha_t,seed,inner_num,timing_tradeoff,td_place_exp_first,td_place_exp_last,place_chan_width,recompute_crit_iter	For optimizing the execution of simulated annealing of placement process in VPR.
routing	initial_pres_fac,pres_fac_mult,acc_fac,bb_factor,base_cost_type,astar_fac,max_criticality,bend_cost,first_iter_pres_fac	Mainly used for optimizing the use of hardware resources during routing in order to prevent the abuse of hardware resources.

4.1 Explore CFA submodel

For simplifying the model construction, we construct the submodels firstly. When the submodels' fitness meets the requirements, then they are integrated together.

In the analysis of section 2.1, the pack, placement and routing of the VTR are divided, and these three steps are separately modeled to construct CFA submodels, finally the fitness of each submodel is evaluated. The 25 variables in Table 1 are divided based on steps, and each step is assigned a latent variable, which represents the intermediate effect of the corresponding step on the final execution result. The parameters in Table 1 are observed parameters that are combined to produce a latent variable.

4.2 Initialize structural submodel

The structural submodel is used to represent the relationship between the latent variables and it is initialized based on the CFA submodel constructed in section 4.1. The latent variables proposed in section 3.1 are the factors that cause the routing failure. The structural submodel is initialized by analyzing the relationship between these latent variables and the routing failure. Since the initialized structural submodel is determined according to the existing experience, there is also a problem of insufficient fitness. When the fitness of the model is too small, the relationship between the latent variables is changed, and the fitness of the model is evaluated again. After the fitness meets the requirements, the structural submodel is finally determined.

5. Results and analysis

5.1. Model analysis

According to the method mentioned in the third section, after trying multiple models, the number of latent variables is determined to be three, representing the three steps of pack, placement and routing. The constructed submodel is shown in Fig. 2 where circles e1-e22 represent errors of various parameters and ellipses represent latent variables. Through the analysis of model fitness by AMOS, the GFI and AGFI fitness index reaches 0.995, and the RMSEA is less than 0.02. When GFI, AGFI, CFI and RNI are greater than 0.95 and the RMSEA is less than 0.06, the proposed model can be considered acceptable¹². The model has met these requirements.

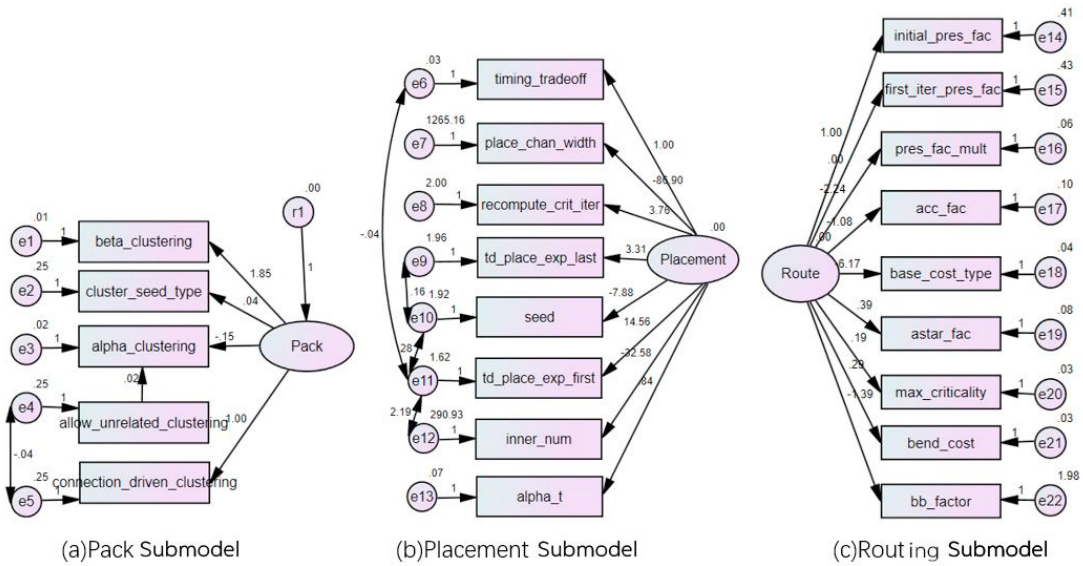


Fig. 2. CFA Submodels

5.2. Benchmarks used in experiments

Benchmark is a test case suite for evaluating the performance of software or hardware products. The benchmarks used in this paper are in the VTR project. These benchmarks are written in Verilog and from Opencores and SCU(Santa Clara University)^{1,4}. The applications of these circuits cover a wide range of fields and are highly representative. Table 2 shows details about these benchmarks.

5.3. Improvements to DATuner

The model is used to guide the exploration of the parameter space. When performing parameter space exploring, the node is deleted in the space partition tree³ for the parameters that satisfy the regression relationship in Fig. 2, thus filtering out the parameter combinations that may cause the routing failure.

Table 2. Benchmark description in VTR.

Circuits	LUT	Description	Number of dataset
Orl200	3075	RISC processor	1300
Sha	2001	Cryptography	1500
MkSMAAdapter4B	1960	Packet Processing	1400
Raygentop	1884	Ray Tracing	1500
Stereovision0	9567	Computer Vision	1000
Blob_merge	8067	Image Processing	1200
Boundtop	3053	Ray Tracing	1300

According to the correlation between the parameters obtained in section 4.1, in the process of parameter space search, in the original DATuner that the parameter space has not executed a prior pruning, DATuner is executed to generate 400 pieces of data. In addition, using the structural equation model derived from Fig. 2, a prior pruning is performed during the execution of DATuner, and 400 pieces of data are generated in the same environment. Figure 3 shows a comparison of runtime generating these 400 pieces of data. Note that the run time in Figure 3 and Table 3

is measured in hours. As can be seen from Fig. 3, for the benchmarks listed in Table 2, after using the structural equation model for prior pruning, the DATuner's running time is reduced about 44%.

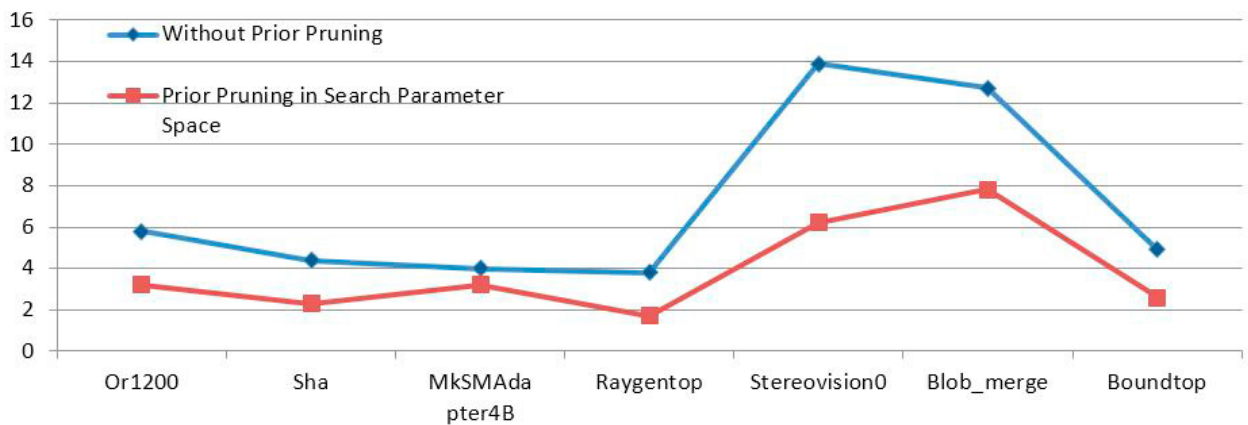


Fig. 3. Running Time Comparison Chart

6. Conclusion and future work

In this paper, we have proposed the prior pruning in autotuning base on structural equation model and do improvements to DATuner. Compared with the DATuner that did not perform a prior pruning, the running time was greatly shortened, and when the same amount of data was generated, the DATuner's running time was reduced by 44%, which improved the execution efficiency of the program.

Since the structural equation model can only analyze the linear relationship between variables. Therefore, future research can explore the possible nonlinear relationships between variables.

Acknowledgements

Thanks to Teacher Luo Guojie of Peking University for his help.

References

- ROSE J, LUU J, YU C W, et al. The VTR project: architecture and CAD for FPGAs from verilog to routing[C]//Proceedings of the ACM/SIGDA international symposium on Field Programmable Gate Arrays. CA: ACM, 2012: 77-86.
- XU CHANG, LIU GAI, ZHAO RITCHIE et al. A Parallel Bandit-Based Approach for Autotuning FPGA Compilation [C]// 2017 ACM/SIGDA International Symposium: 2017 Association for Computing Machinery Special Interest Group on Design Automation. CA: ACM, 2017:157-166.
- LIU H Y, CARLONI L P. On learning-based methods for design-space exploration with high-level synthesis[C]//Proceedings of the 50th annual design automation conference. TX: ACM, 2013: 50-56.
- LUU J, GOEDERS J, WAINBERG M, et al. VTR 7.0: Next generation architecture and CAD system for FPGAs [J]. ACM Transactions on Reconfigurable Technology and Systems (TRETS), 2014, 7(2): 6-35.
- ANSEL J, KAMIL S, VEERAMACHANENI K, et al. Opentuner: An extensible framework for program autotuning[C]//Proceedings of the 23rd international conference on Parallel architectures and compilation. Edmonton: ACM, 2014: 303-316.
- JAMIESON P, KENT K B, GHARIBIAN F, et al. Odin ii-an open-source verilog hdl synthesis tool for cad research[C]//2010 18th IEEE Annual International Symposium on Field-Programmable Custom Computing Machines. North Carolina: IEEE, 2010: 149-156.
- BRAYTON R, MISHCHENKO A. ABC: An academic industrial-strength verification tool[C]//International Conference on Computer Aided Verification. Berlin: Springer, 2010: 24-40.
- NACHIKET K, QUE Y, HARNHUA N et al. Improving Classification Accuracy of a Machine Learning Approach for FPGA Timing Closure [C]// 2016 FCCM: Proceedings of the 24th IEEE International Symposium on Field-Programmable Custom Computing Machines. Washington: IEEE, 2016: 80-83.

9. DAI S, ZHOU Y, ZHANG H, et al. Fast and accurate estimation of quality of results in high-level synthesis with machine learning [C]// Proceedings of the 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). CO: IEEE, 2018: 129-132.
10. USTUN E, XIANG S, GUI J, et al. LAMDA: Learning-Assisted Multi-stage Autotuning for FPGA Design Closure[C]//2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). CA: IEEE, 2019: 74-77.
11. Li D, Yao S, Liu Y H, et al. Efficient design space exploration via statistical sampling and AdaBoost learning [C]// 2016 53rd ACM/EDAC/IEEE Design Automation Conference (DAC). TX: IEEE, 2016:1-6.
12. LUU J, ANDERSON J H, ROSE J S. Architecture description and packing for logic blocks with hierarchy, modes and complex interconnect[C]// Proceedings of the 19th ACM/SIGDA international symposium on Field programmable gate arrays. CA: ACM, 2011: 227-236.